Text-Based Invoice Data Extraction

Objective: To develop a Python program that processes formatted invoice strings, extracts product codes, quantities, and amounts using basic string operations, and identifies malformed invoice records.

1. Computational Thinking

- Decomposition: Divide the task into reading invoice records, extracting details, validating data, and displaying results.
- Pattern Recognition: Identify the common format of invoice records.
- Abstraction: Focus on product code, quantity, and amount.
- Algorithm Design: Follow step-by-step instructions to process invoice data.

2. Input Format

Each invoice record follows this format:

P101,2,500

P102,3,750

P103,1,400

P104,abc,600

P105,2

Here:

- P101 = Product code
- 2 = Quantity
- 500 = Amount

3. Algorithm

1. Start.
2. Enter the number of invoice records.
3. Read each invoice record as a string.
4. Split the string using commas.
5. Check whether the record contains exactly three fields.
6. Extract product code, quantity, and amount.
7. Check whether quantity and amount are valid numbers.
8. If valid, calculate the total amount.

9. If invalid, flag the record as malformed.
10. Display valid invoice details.
11. Display malformed records.
12. Stop.

4. Python Program

n = int(input("Enter number of invoice records: "))valid_records = []malformed = []for i in range(n): record = input("Enter invoice record: ") parts = record.split(",") if len(parts) == 3: code = parts[0].strip() qty = parts[1].strip() amount = parts[2].strip() if code.startswith("P") and code[1:].isdigit() and qty.isdigit() and amount.isdigit(): qty = int(qty) amount = float(amount) total = qty * amount valid_records.append((code, qty, amount, total)) else: malformed.append(record) else: malformed.append(record)print("\n----- INVOICE REPORT -----")for code, qty, amount, total in valid_records: print("Product Code:", code) print("Quantity:", qty) print("Amount:", amount) print("Total:", total) print()print("Malformed Records:", malformed)

5. Sample Input

Enter number of invoice records: 5

P101,2,500

P102,3,750

P103,1,400

P104,abc,600

P105,2

6. Sample Output

----- INVOICE REPORT -----

Product Code: P101

Quantity: 2

Amount: 500.0

Total: 1000.0

Product Code: P102

Quantity: 3

Amount: 750.0

Total: 2250.0

Product Code: P103

Quantity: 1

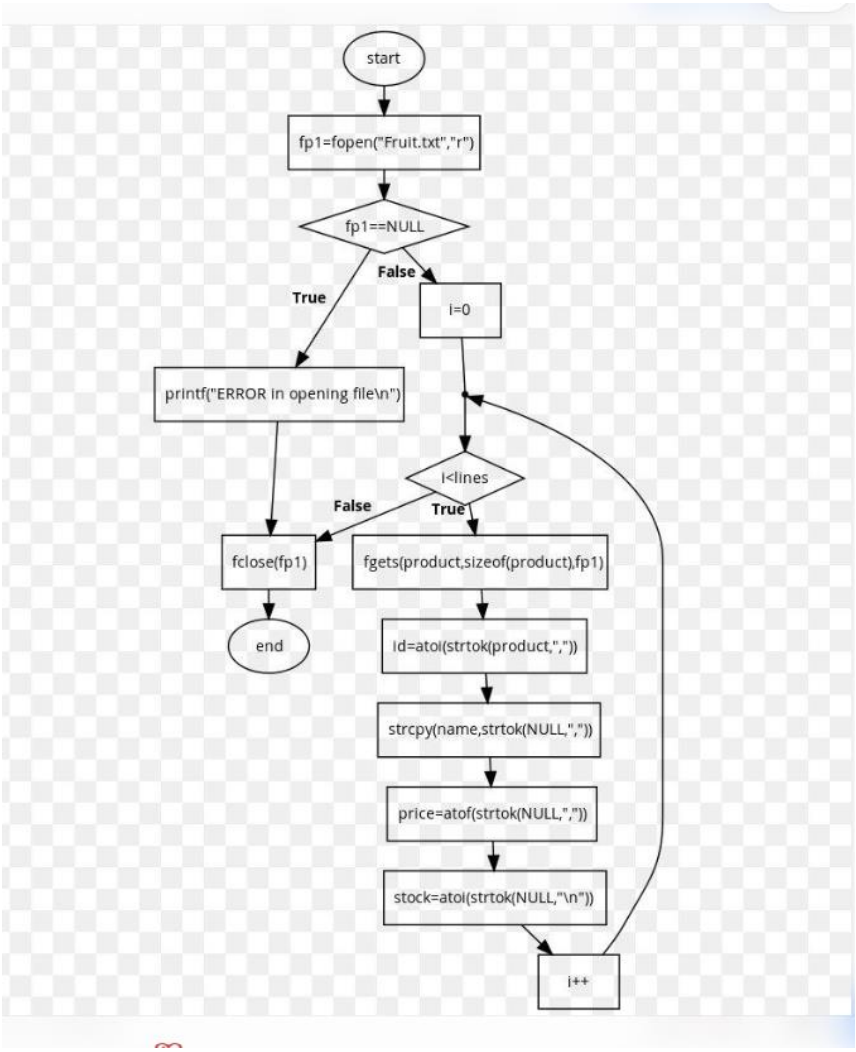
Amount: 400.0

Total: 400.0

Malformed Records:

['P104,abc,600', 'P105,2']

7. Flowchart



8. Python Concepts Used

Concept	Application
Variables	Store product code, quantity and amount
Data types	String, integer, float
Operators	Multiplication and comparison
Conditional statements	Validate invoice records
Loops	Process multiple records
Strings	Read invoice data
String operations	split(), strip(), isdigit(), startswith()
Pattern generation	Display invoice report
Lists	Store valid and malformed records
Tuples	Store product details
Packing	Group product details into tuples

9. Pseudocode

```
START

INPUT n
CREATE valid_records
CREATE malformed_records

FOR i = 1 TO n
    INPUT record
    SPLIT record using comma

    IF number of fields = 3 THEN
        EXTRACT code, quantity, amount
```

```
        IF all fields are valid THEN
            CALCULATE total = quantity * amount
            STORE details in valid_records
        ELSE
            STORE record in malformed_records
        END IF
    ELSE
        STORE record in malformed_records
    END IF
END FOR

DISPLAY valid_records
DISPLAY malformed_records

STOP
```

10. Conclusion

This program extracts product codes, quantities, and amounts from invoice strings. It identifies incorrect records and generates an invoice report using basic Python concepts such as strings, loops, conditions, lists, and tuples.